TECHNISCHE UNIVERSITÄT WIEN

ILSB Institute of Lightweight Design and Structural Biomechanics

317.047 Computational Bionics

Control System

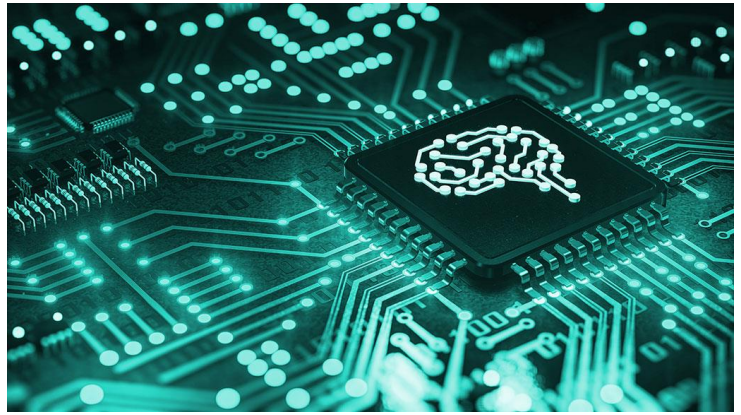
Dipl.-Ing. Dr. techn. Harald Sima

Central nervous system/control

The control system and the associated control loops breathe life into every machine.

The central **nervous system** (CNS), i.e. the analogue to the **main processor**, receives information from the environment and from inside the body, processes it and sends commands to the body regions.

Example:
350,000 nerves emerge from the spinal cord to innervate the upper limbs. Of these, only 10 % are motor neurons.

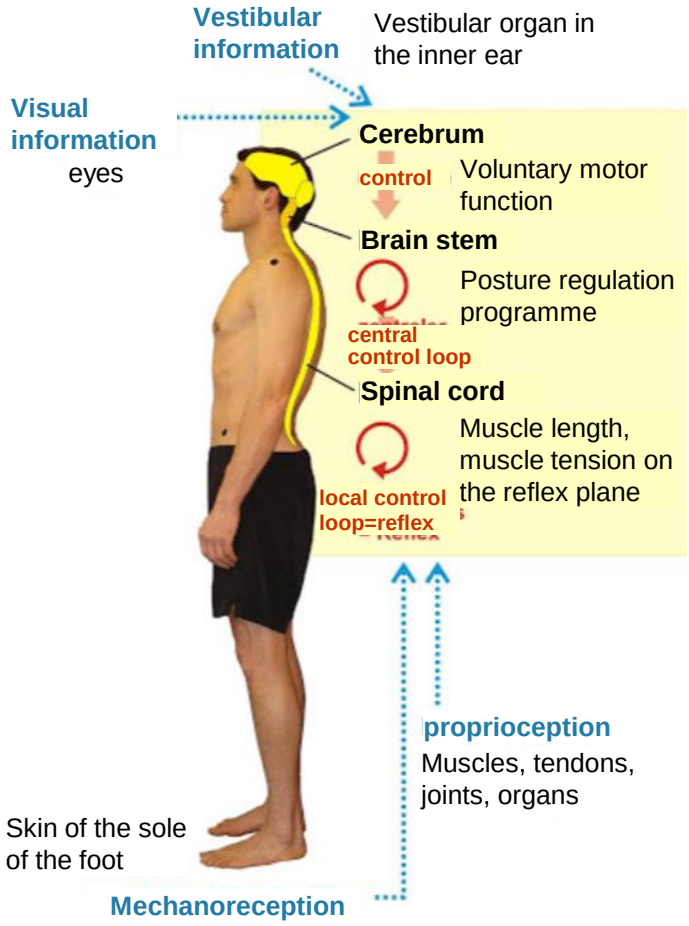


Nervous system/control
Sensorimotor control loop

Commands from the CNS are often still at an **abstracted level** and are converted in the PNS into **direct control**, e.g. of the motor units. Technically speaking, this can reduce the density of information flow over longer distances.

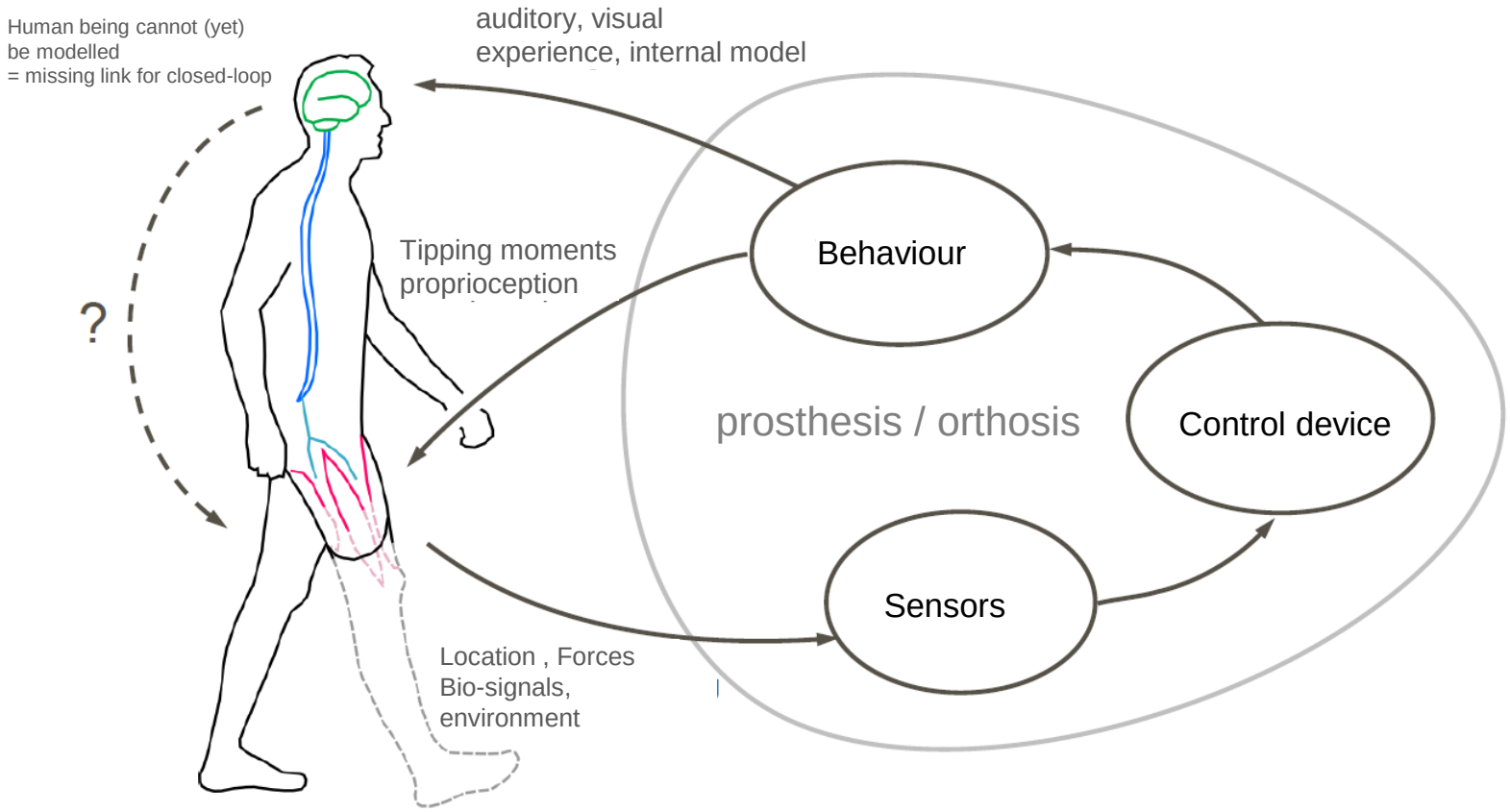
There are also **local control loops** that operate without direct involvement of the CNS.
This has the same reason in humans as in technical applications. It allows a much **faster response** time.

This results in a distributed system.

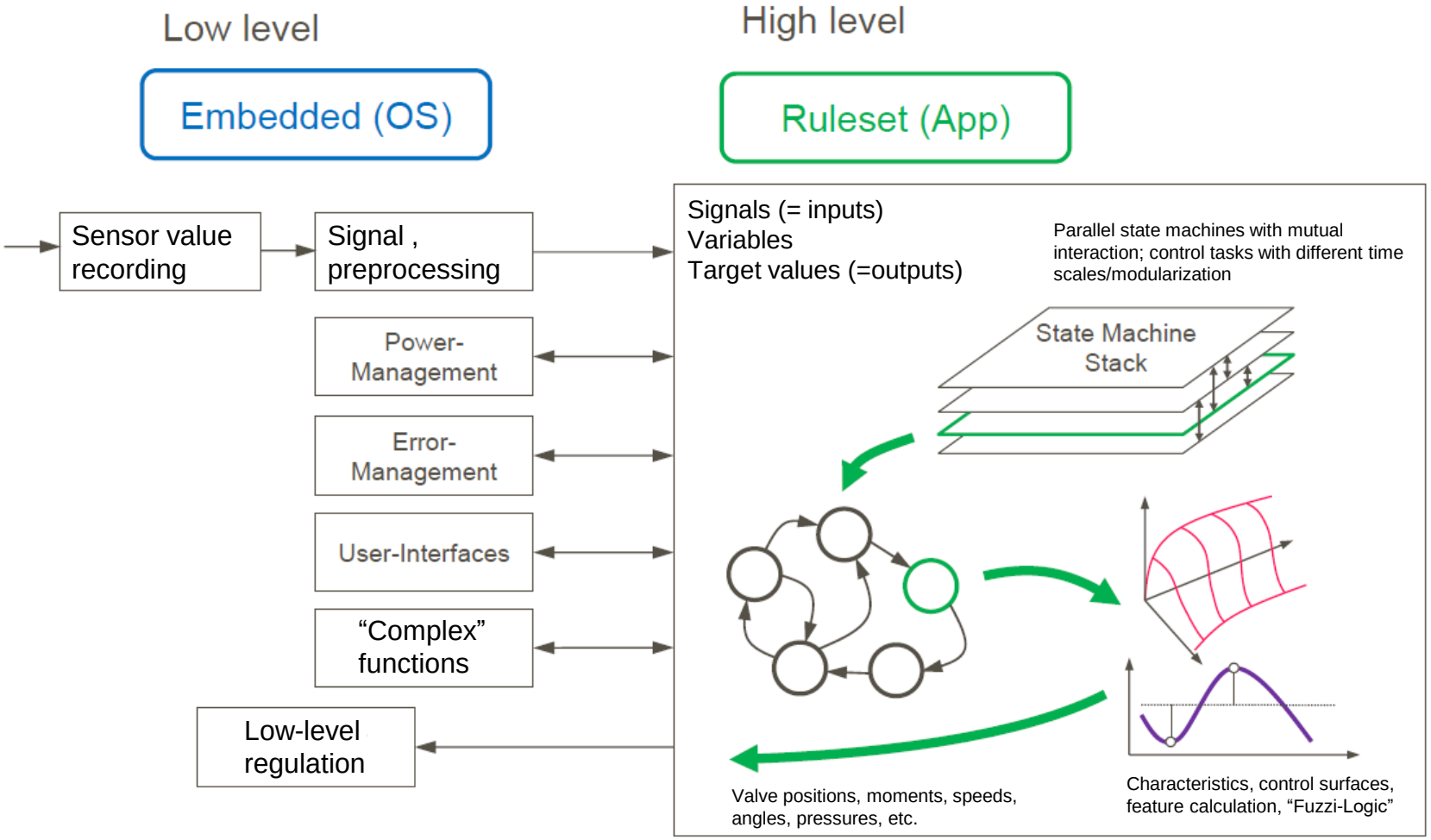


Human in the Loop

- The user and its prostheses build a **closed lop control system** for very diverse applications in the everyday usage.
- The **prosthesis cannot function without its user**. It requires interaction with the user.



High-/Low Level Control



High-/Low Level Control

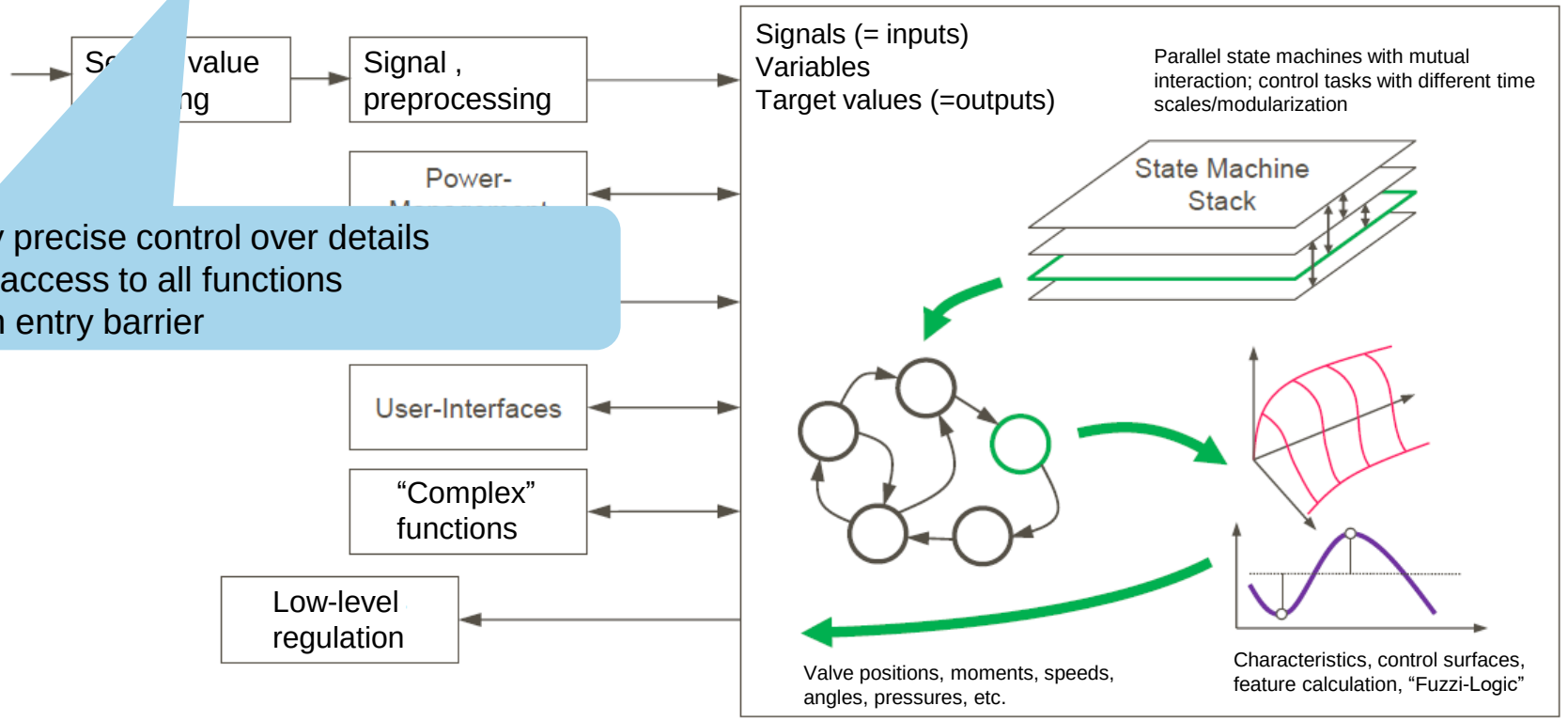
☺ Visual style, very intuitive
☺ Easy programming start
☹ Limited possibilities

Low level

Embedded (OS)

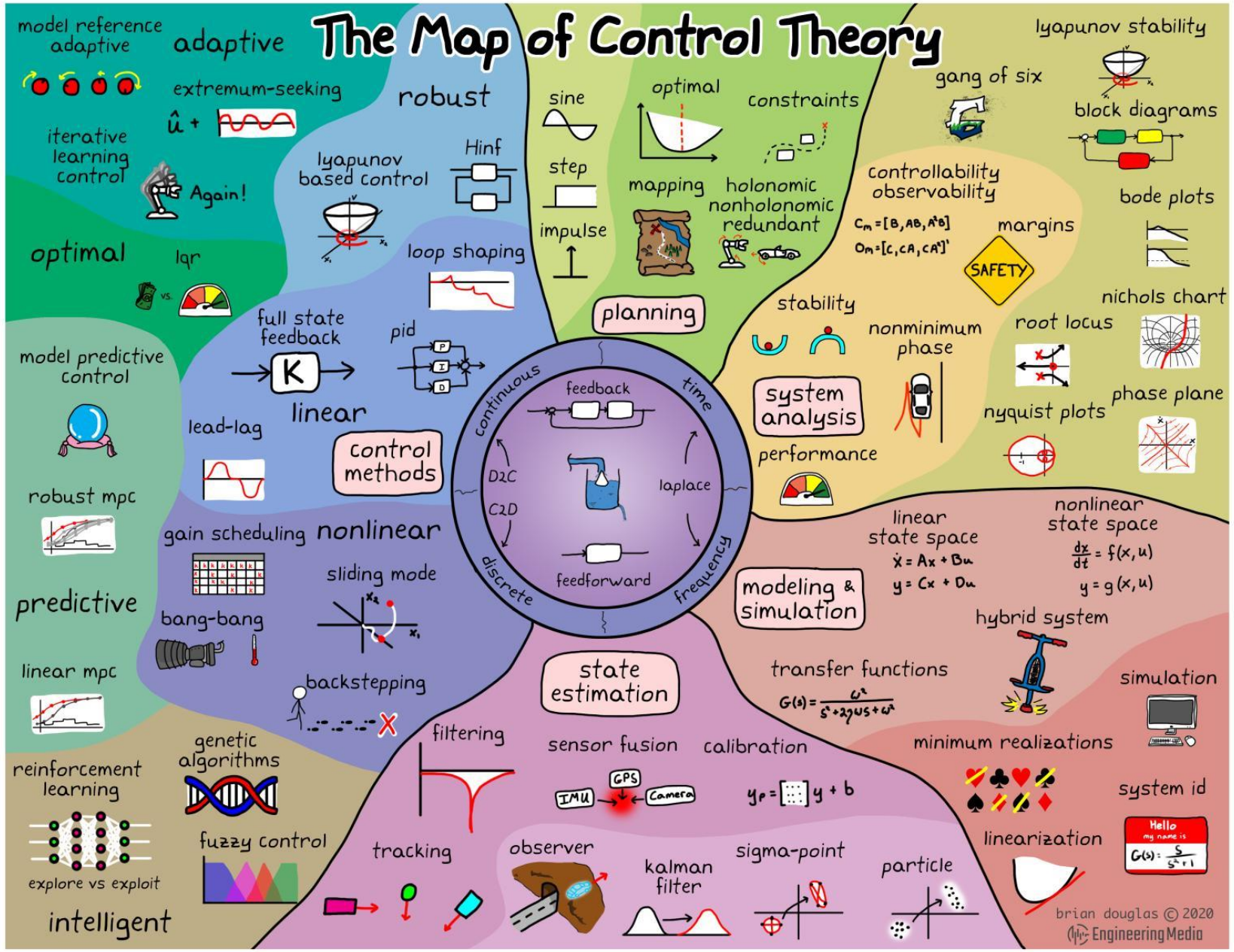
High level

Ruleset (App)



☺ Very precise control over details
☺ Full access to all functions
☹ High entry barrier

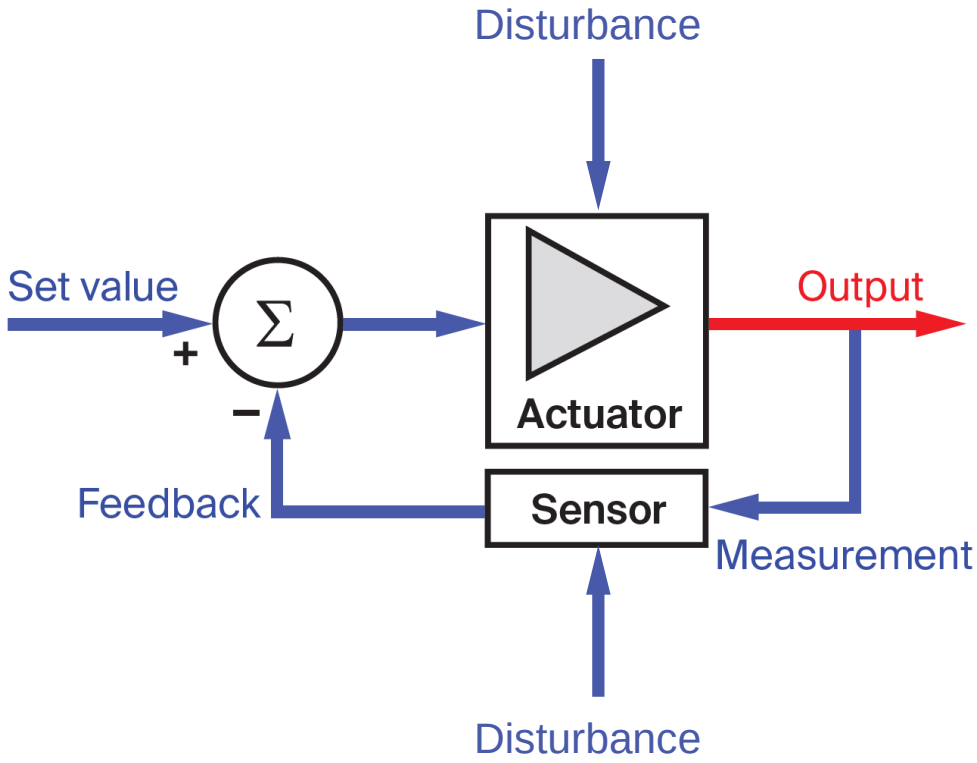
Low level control


The Map of Control Theory
adaptive
model reference adaptive
extremum-seeking
iterative learning control
Again!
robust
lyapunov based control
Hinf
loop shaping
optimal
lqr
vs.
full state feedback
K
pid
P
I
D
linear
lead-lag
control methods
model predictive control
robust mpc
gain scheduling
nonlinear
sliding mode
predictive
bang-bang
linear mpc
backstepping
reinforcement learning
genetic algorithms
explore vs exploit
fuzzy control
intelligent
sine
step
impulse
optimal
constraints
mapping
holonomic
nonholonomic
redundant
planning
lyapunov stability
gang of six
block diagrams
controllability
observability
$C_m = [B, AB, A^2B]$
$O_m = [C, CA, CA^2]^T$
margins
SAFETY
bode plots
stability
nonminimum phase
root locus
nichols chart
system analysis
nyquist plots
phase plane
performance
continuous
discrete
time
frequency
feedback
feedforward
D2C
C2D
laplace
linear state space
$\dot{x} = Ax + Bu$
$y = Cx + Du$
nonlinear state space
$\frac{dx}{dt} = f(x,u)$
$y = g(x,u)$
modeling & simulation
hybrid system
transfer functions
$G(s) = \frac{\omega^2}{s^2 + 2\zeta\omega s + \omega^2}$
simulation
minimum realizations
system id
Hello
my name is
$G(s) = \frac{s}{s+1}$
linearization
state estimation
filtering
sensor fusion
GPS
IMU
Camera
calibration
$y_p = \boxed{\ } y + b$
tracking
observer
kalman filter
sigma-point
particle
brian douglas © 2020
Engineering Media

Basics of control theory

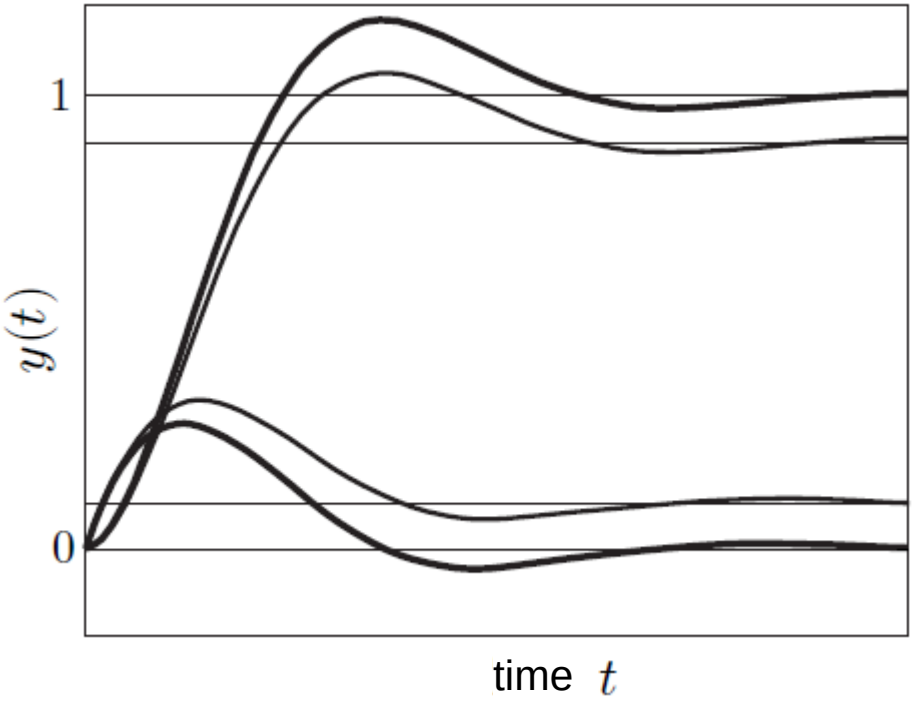
- The task of a control system is to **maintain physical variables in a technical system**, at a constant value or to give them a certain trajectory. This should be done with the greatest possible accuracy and specified dynamics.

- Definition according to DIN 19226
  Control is a process in which a variable, the variable to be controlled (controlled variable),continuously recorded, compared with another variable, the reference variable (the setpoint), and is influenced in the sense of an adjustment to the reference variable (the setpoint). Characteristic for closed-loop control is characterized by the closed-loop action in which the controlled variable control loop continuously influences itself.



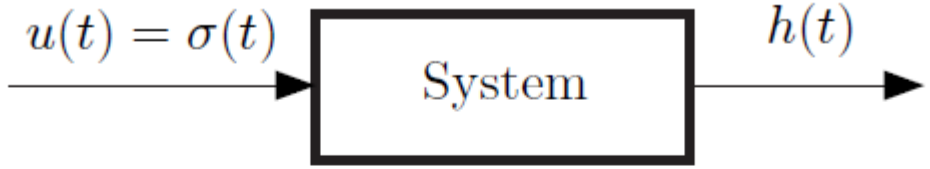
Requirements for a control system

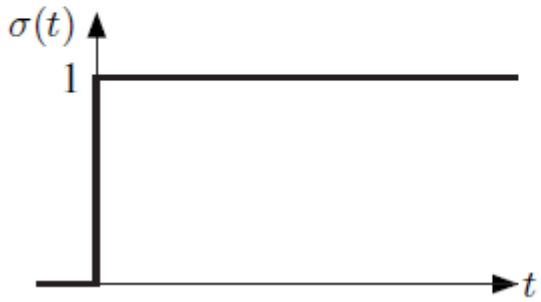
- Requirements for a control system
- **Stability**:
  all dynamic processes after a setpoint and disturbance variable change in the control loop must decay.
- **Steady-state accuracy**:
  the steady-state control error must not exceed a specified value after setpoint changes and after the occurrence of disturbances.
- **Speed:**
  the dynamic processes in the event of a setpoint change and after the occurrence of a fault must be completed after an acceptable time.
- **Sufficient damping**:
  The response of the control system to a change in the reference variable or to the occurrence of a disturbance must be disturbance must be sufficiently damped.

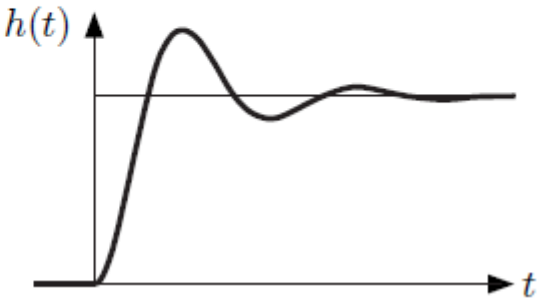


Transfer function

- In system theory, the transfer function mathematically describes the **relationship between the input and output** signal of a linear dynamic system.
- In case of a nonlinear system, we can **linearize** it in its operation point for further investigation.
- To characterize and compare the dynamic behavior of the system responses often three special test functions are often used.
  - unit step function σ(t)
  - impulse function (Dirac delta-function) δ(t)
  - unit ramp function ρ(t).







Transfer function

- A linear dynamic system is described by the differential equation

$$a_n y^{(n)}(t) + \cdots + a_1 \dot{y}(t) + a_0 y(t) = b_0 u(t) + b_1 \dot{u}(t) + \cdots + b_m u^{(m)}(t) \quad m \leq n$$

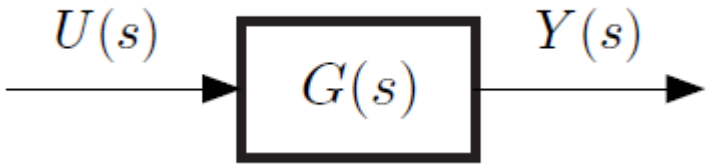
- Assuming vanishing initial conditions, the Laplace transformation follows to

$$\left(a_n s^n + \cdots + a_2 s^2 + a_1 s + a_0\right) Y(s) = \left(b_m s^m + \cdots + b_2 s^2 + b_1 s + b_0\right) U(s)$$

- We rewrite and get the transfer function

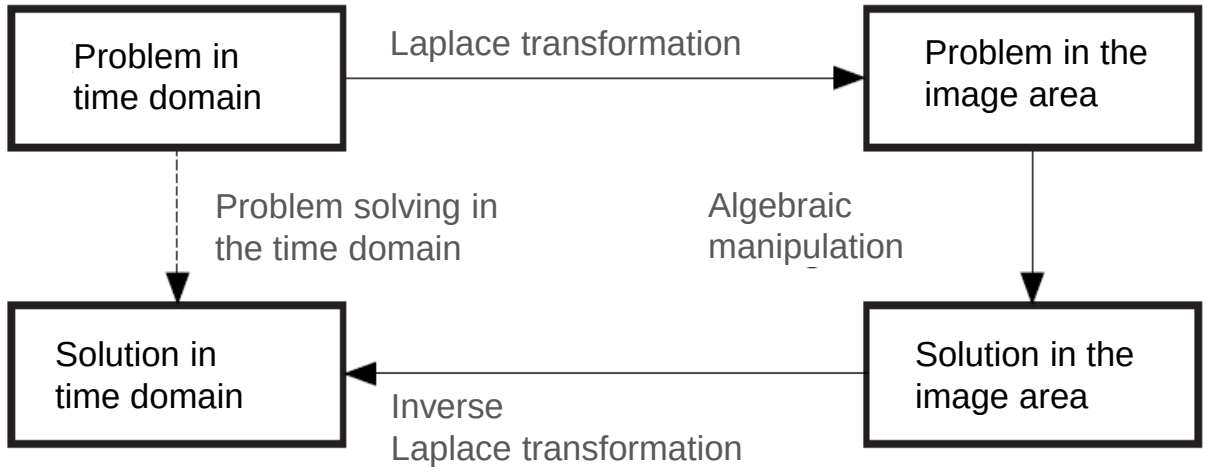
$$G(s) = \frac{Y(s)}{U(s)} = \frac{b_m s^m + \cdots + b_2 s^2 + b_1 s + b_0}{a_n s^n + \cdots + a_2 s^2 + a_1 s + a_0} = \frac{B(s)}{A(s)}$$

$$Y(s) = G(s)U(s)$$



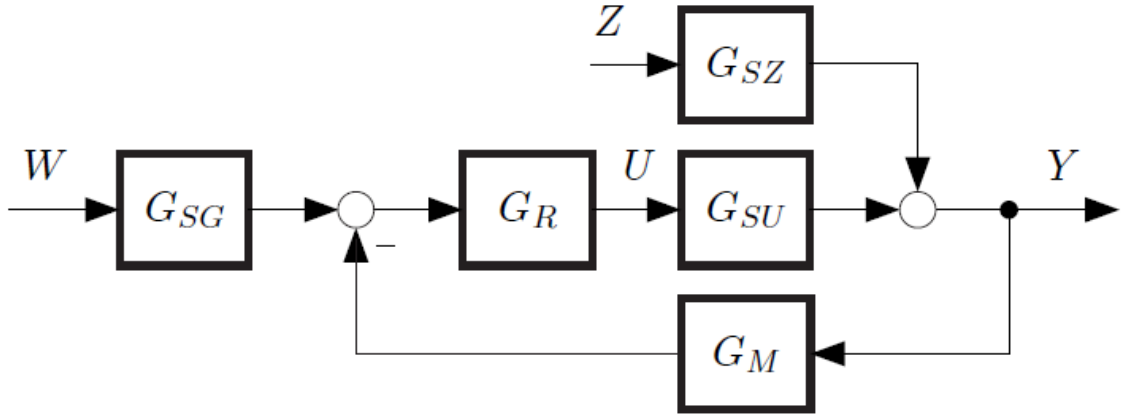
Laplace transformation

- The Laplace transformation is an **integral transform** that transforms a given real-valued function f : R → R into a function F(s) in the complex image domain. The spectral domain or frequency domain.
- The transformation is suitable for **solving linear differential equations** and systems of differential equations with constant coefficients. Differential equations are transformed to algebraic equations in the image domain.
- Often the apparent detour via the Laplace transformation can mean a simpler solution than solving the problem directly in the time domain.
- The Laplace transform plays a particularly important role in control engineering, mainly due to the convolution theorem.
- Complex overall systems can also be simplified and modified very easily after the introduction of the term transfer function.



Single input single output controller

- GSG(s) setpoint transfer function
- GR(s) controller transfer function
- GSU(s) actuator transfer function
- GSZ(s) disturbance transfer function
- GM(s) measuring device transfer function



- Open loop transfer function $G_o(s) = G_R(s)G_{SU}(s)G_M(s)$
- Closed loop transfer function $Y = G_{SG}\frac{G_R G_{SU}}{1+G_o}W + \frac{G_{SZ}}{1+G_o}Z$
- Set point transfer function $Z = 0: \quad G_{UW} = \frac{U}{W} = \frac{G_{SG}G_R}{1+G_o}$
- Disturbance transfer function $W = 0: \quad G_{UZ} = \frac{U}{Z} = -\frac{G_R G_M G_{SZ}}{1+G_o}$

PID controller

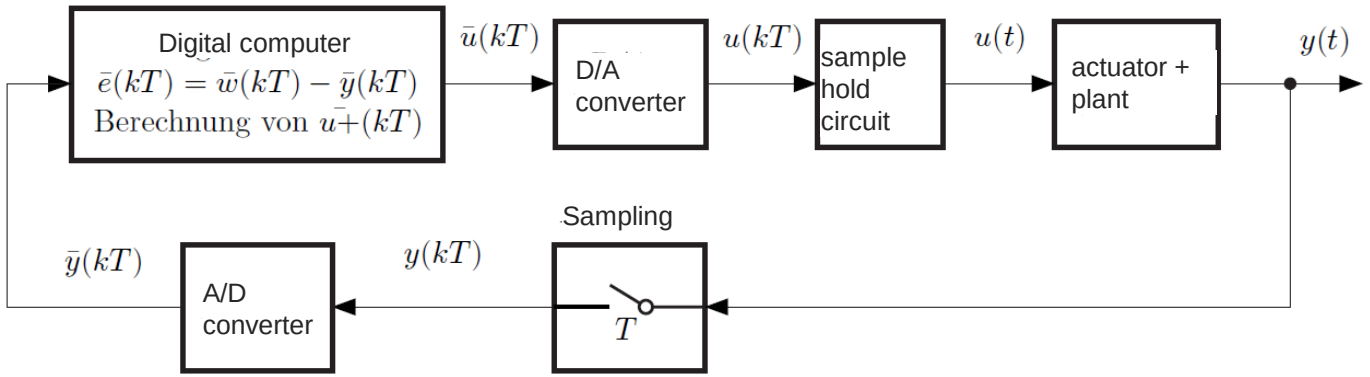
- We consider the classic PID controller (Proportional-Integrating-Differential controller)
  - K_P is the controller gain
  - K_I the integrator gain
  - K_D the differentiator gain.

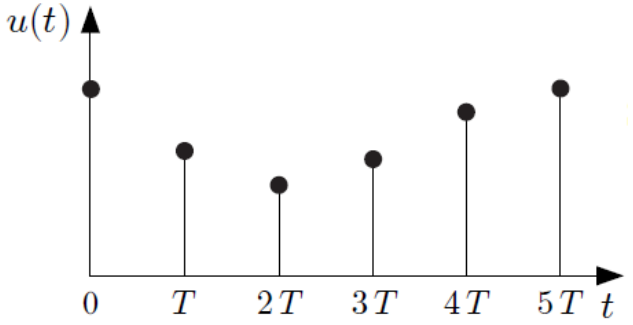
$$u(t) = K_P e(t) + K_I \int e(t)\mathrm{d}t + K_D \frac{\mathrm{d}e(t)}{\mathrm{d}t}$$

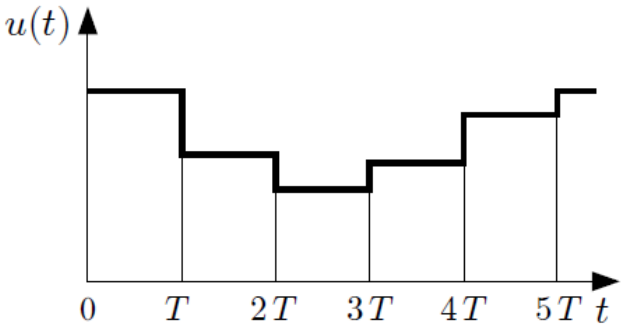
$$G_R(s) = K_P + \frac{K_I}{s} + K_D s = \frac{K_D s^2 + K_P s + K_I}{s}$$

Digital PID controller

- In realty we had to deal with **digital controllers**
- The controlled variable **y(t) is periodically sampled** synchronously at the sampling frequency f = 1/T, with aconstant sampling time T.
- The analog analog-to-digital converter (ADC) convert the **time-discrete signal** sequence y(kT) in a binary signal sequence y*(kT)
- the process computer, generates the reference variable w*(kT), compute the control error e*(kT) = w*(kT) – y*(kT) and the binary control signal u*(kT) by the control algorithm
- The digital control signal is then converted into the analog control value sequence u(kT) with by the digital-to-analog converter (DAW) and stored over the following sampling interval.







Digital PID controller

- To realize the digital controller we approximate the control equation

$$u(t) = K_P e(t) + K_I \int e(t)\,dt + K_D \frac{de(t)}{dt} \qquad \int_0^t e(\tau) \approx T \sum_{i=0}^{k-1} e(iT)$$

$$u(kT) = K_P e(kT) + K_I T \sum_{i=0}^{k-1} e(iT) + \frac{K_D}{T}\left(e(kT) - e((k-1)T)\right)$$

$$u((k-1)T) = K_P e((k-1)T) + K_I T \sum_{i=0}^{k-2} e(iT) + \frac{K_D}{T}\left(e((k-1)T) - e((k-2)T)\right)$$

subtract

$$u(kT) = u((k-1)T) + K_P e(kT) + \left(K_I T - 2\frac{K_D}{T} - K_P\right) e((k-1)T) + \frac{K_D}{T} e((k-2)T)$$

Control parameter optimisation

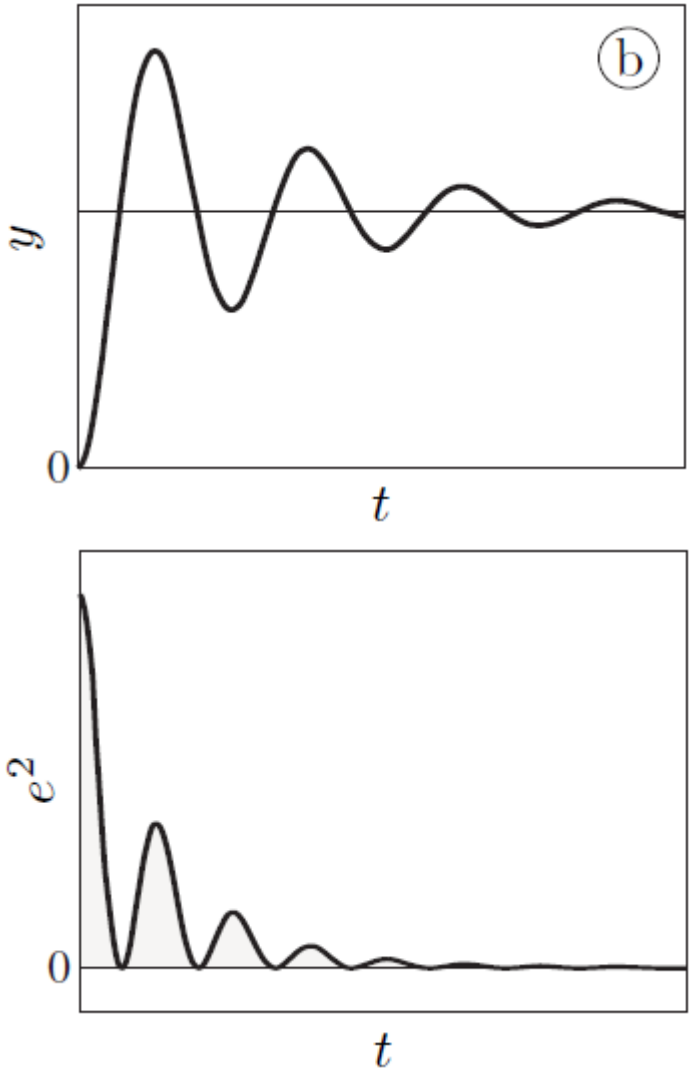
- A very universal methodic for **determining the optimal parameters** of a controller with an already defined structure by **minizine a scalar cost functional**.
- Often an integral criterium of the quadratic control error is used.

$$J = \int_0^\infty e(t)^2 \, dt \quad \Rightarrow \quad \min$$

- We can **use the transfer function** to compute the parameter

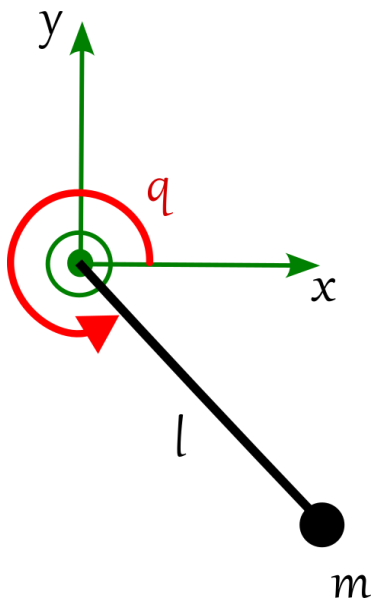
$$J = \int_0^\infty e^2(t) \, dt = \frac{1}{2\pi} \int_{-\infty}^\infty |E(j\omega)|^2 \, d\omega \qquad E(j\omega) = \frac{1}{1 + G_o(j\omega)} \frac{1}{j\omega}$$

- Or for complex systems **use the simulation model for optimization**.
  - select initial parameters
  - excite the simulation with a step change of the set point
  - minimize the cost functional by variation of the control parameters



Controller Example

- Our example is inspired by the “Honda Walking Asist”
- We reuse our pendulum model for the leg.
- We fix the knee and concentrate the mass of the leg in in its center of mass.
- We ignore ground contact.

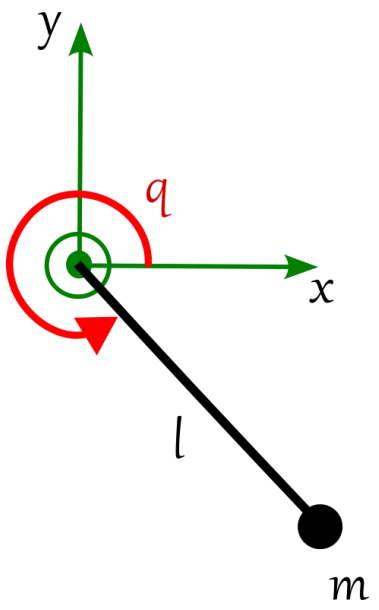


Controller Example – ideal drive and ideal sensor

- The hip movement is supported by an electrical drive
- As a reference trajectory we use a cosine with a frequency of 1 Hz from q = -110° to q = -70°

```
# Pendulum parameters
g = 9.81  # Gravitational acceleration in m/s^2
L = 1.0 * 0.4  # Pendulum length in m
m = 80.0 * 0.2  # Pendulum mass in kg
```

```
# Desired trajectory: sinusoidal input for the angle
def reference_trajectory(t):
    return -np.pi/2 - 20*np.pi/180 *np.cos(1 * t)

# Function that defines the system of differential equations
def pendulum(t, y, M):
    omega, q = y
    domega = - (g / L) * np.cos(q) + M / (m * L**2)
    dq = omega
    return [domega, dq]
```



Controller Example – ideal drive and ideal sensor

- We use a digital PID controller with a sampling Time $dt_{control} = 0.1$ s
- The simulation shall be solved at the timesteps $dt_{sim} = 0.01$ s
- We define the controller as class so we can store the previews values e_{k-1}, e_{k-2} and u_{k-1} in the controller

```python
# PID Controller
class PIDController:
    def __init__(self, Kp, Ki, Kd, T):
        self.Kp = Kp
        self.Ki = Ki
        self.Kd = Kd
        self.T = T
        self.e_km1 = 0
        self.e_km2 = 0
        self.u_km1 = 0

    def compute_control(self, reference, state):
        e_k = reference - state
        a0 = (self.Kp + self.Kd/self.T)
        #a0 = (self.Kp + self.Kd/self.T + self.Ki*self.T)
        a1 = ( self.Ki*self.T - 2*self.Kd/self.T - self.Kp )
        #a1 = ( - 2*self.Kd/self.T - self.Kp )
        a2 = self.Kd/self.T
        u_k = self.u_km1 + a0* e_k + a1* self.e_km1 + a2* self.e_km2
        self.u_km1 = u_k
        self.e_km2 = self.e_km1
        self.e_km1 = e_k

        return u_k
```

Controller Example – ideal drive and ideal sensor

```python
def simulation(params,PLOT):

    Kp, Ki, Kd = params
    controller = PIDController(Kp, Ki, Kd, dt_control)

    # Initialize lists for overall results
    t_total = []
    y_total = []
    torque_array = []
    ref_array = []

    current_state = initial_state

    for i in range(num_steps):
        current_time = t_control[i]

        # Desired trajectory
        reference = reference_trajectory(current_time)

        # Calculate control input
        torque = controller.compute_control(reference, current_state[1])
```

```python
## Simulation

# initial value
initial_state = [omega_0, q_0]
t_control = np.arange(0, t_end, dt_control)
num_steps = len(t_control)


t_sol, y_sol, ref_sol, torque_sol = simulation([Kp, Ki, Kd], True)
```

- To solve our digital system we split into timesteps of $dt_{control}$ with a for loop.
- We initialize the controller with its parameter
- In each loop we compute the control variable, in our case the torque of the drive.
- The control variable is constant over the sampling interval.
- For now, we don’t model the drive and assume an ideal torque.

Controller Example – ideal drive and ideal sensor

- The next step in our loop is to solve system of ODEs for our pendulum.
- The time span correspond to the actual sampling interval
- We force the solver to generate the simulated output values at a time grid with dt_{sim} by t_eval and dense_output
- We save the last state of the solution as current_state to start the next period at this point

```
# Define the time span for the current controller interval
t_span = [current_time, min(current_time + dt_control, t_end)]

# Calculate the time points for t_eval (each simulation time step)
t_eval = np.linspace(t_span[0], t_span[1], int((t_span[1] - t_span[0]) / dt_sim) + 1)

# Solve the ODE for the current interval, passing the torque
sol = solve_ivp(system_equations, t_span, current_state, args=(torque,), method='RK45', t_eval=t_eval, dense_output=True)

# The final state of the current interval becomes the initial state for the next interval
current_state = sol.y[:, -1]
```

Controller Example – ideal drive and ideal sensor

- In preparation to add additional sub models, like the drive, we wrap the pendulum function into a function system_equation which is called by the solver.
- We has to hand over the torque as a parameter because its note a state of our ODE system. This will be done by args
- Mark the parameter for the solver

```python
# Function that defines ODEs of the pendulum
def pendulum(t, y, M):
    omega, q = y
    domega = - (g / L) * np.cos(q) + M / (m * L**2)
    dq = omega
    return [domega, dq]

# Function of the complete system for the solver
def system_equations(t, y, args):
    dydt = pendulum(t, y, args)
    return dydt
```

```python
# Solve the ODE for the current interval, passing the torque
sol = solve_ivp(system_equations, t_span, current_state, args=(torque,), method='RK45', t_eval=t_eval, dense_output=True)
```

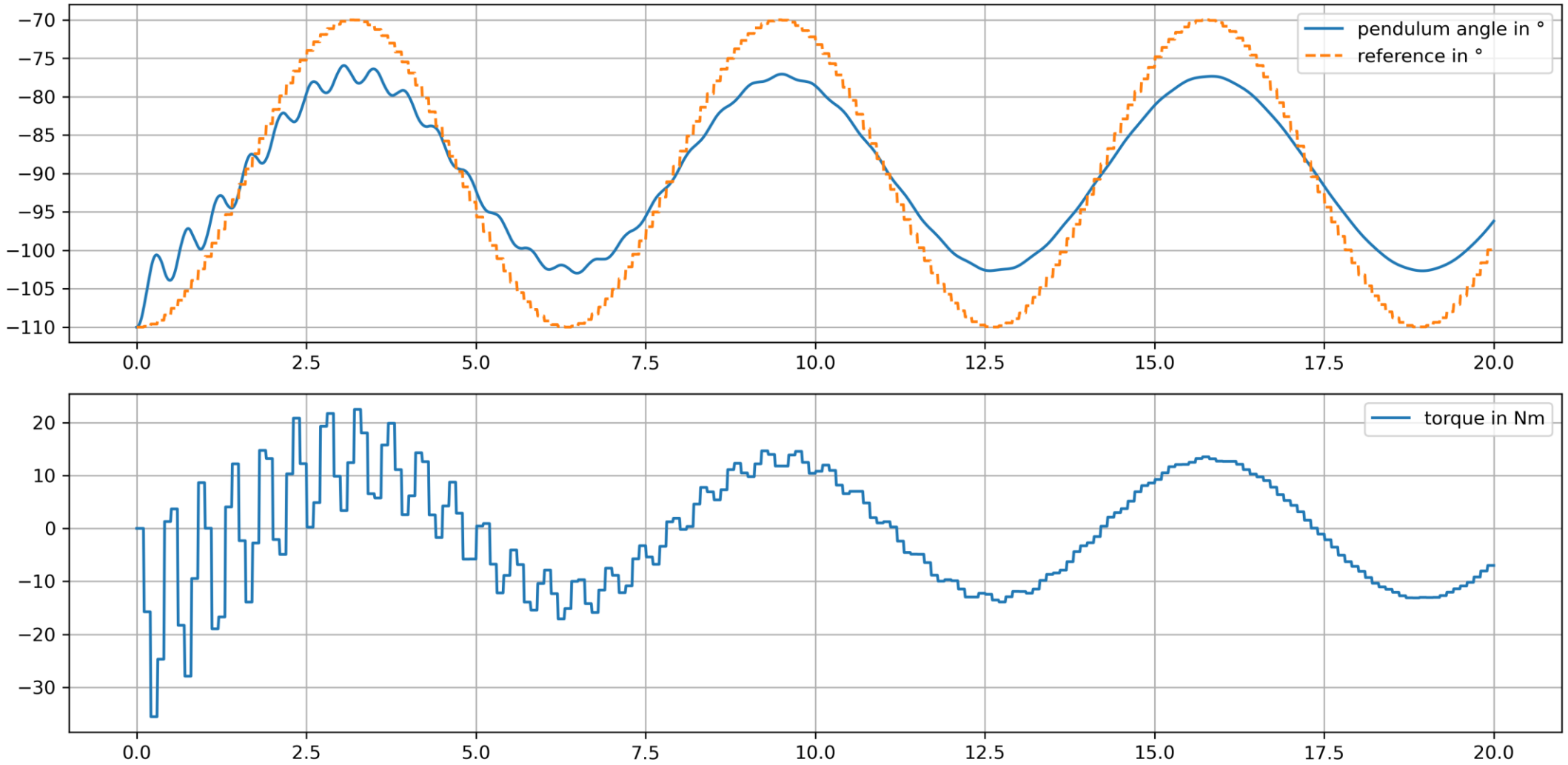
Controller Example – ideal drive and ideal sensor

- The last step in our loop to cat the result from each solver run to the total result.
- The last value of each sampling period is omitted so that there is no overlap

```
# Store time and state values, but without duplicating the first value
if len(t_total) == 0:  # If it's the first iteration
    t_total.extend(sol.t)
    y_total.extend(sol.y.T)  # .T for row-wise
    ref_array.extend(np.ones(len(sol.t))*reference)
    torque_array.extend(np.ones(len(sol.t))*torque)
else:
    t_total.extend(sol.t[1:])  # Only add new time points (without the first)
    y_total.extend(sol.y[:, 1:].T)  # Only add new states (without the first)
    ref_array.extend(np.ones(len(sol.t)-1)*reference)
    torque_array.extend(np.ones(len(sol.t)-1)*torque)
```

```
return t_total, y_total, ref_array, torque_array
```

Controller Example – ideal drive and ideal sensor



Controller Example

- Until now e just choose our control parameter by trial an error. Let's look how we can optimize them:
- Define a cost function.
- Additional to the square of the control error we can consider additional aspects like torque, energy consumption, …
- Be carful choosing weighting factors between all your considered aspects to gat good result.

```python
# Objective function for optimization
def cost_function(params):

    #print(params)
    t_sol, y_sol, ref_sol, torque_sol = simulation(params, False)

    total_error = np.sum((np.array(y_sol)[:, 1] - np.array(ref_sol))**2)
    total_torque = np.sum((np.array(torque_sol))**2)
    #print(total_error + 0*total_torque)

    # Objective function: combination of squared error and squared torque
    return total_error + 0 * total_torque
```

$$J = \int_0^\infty e(t)^2 \, dt \quad \Rightarrow \quad \min$$

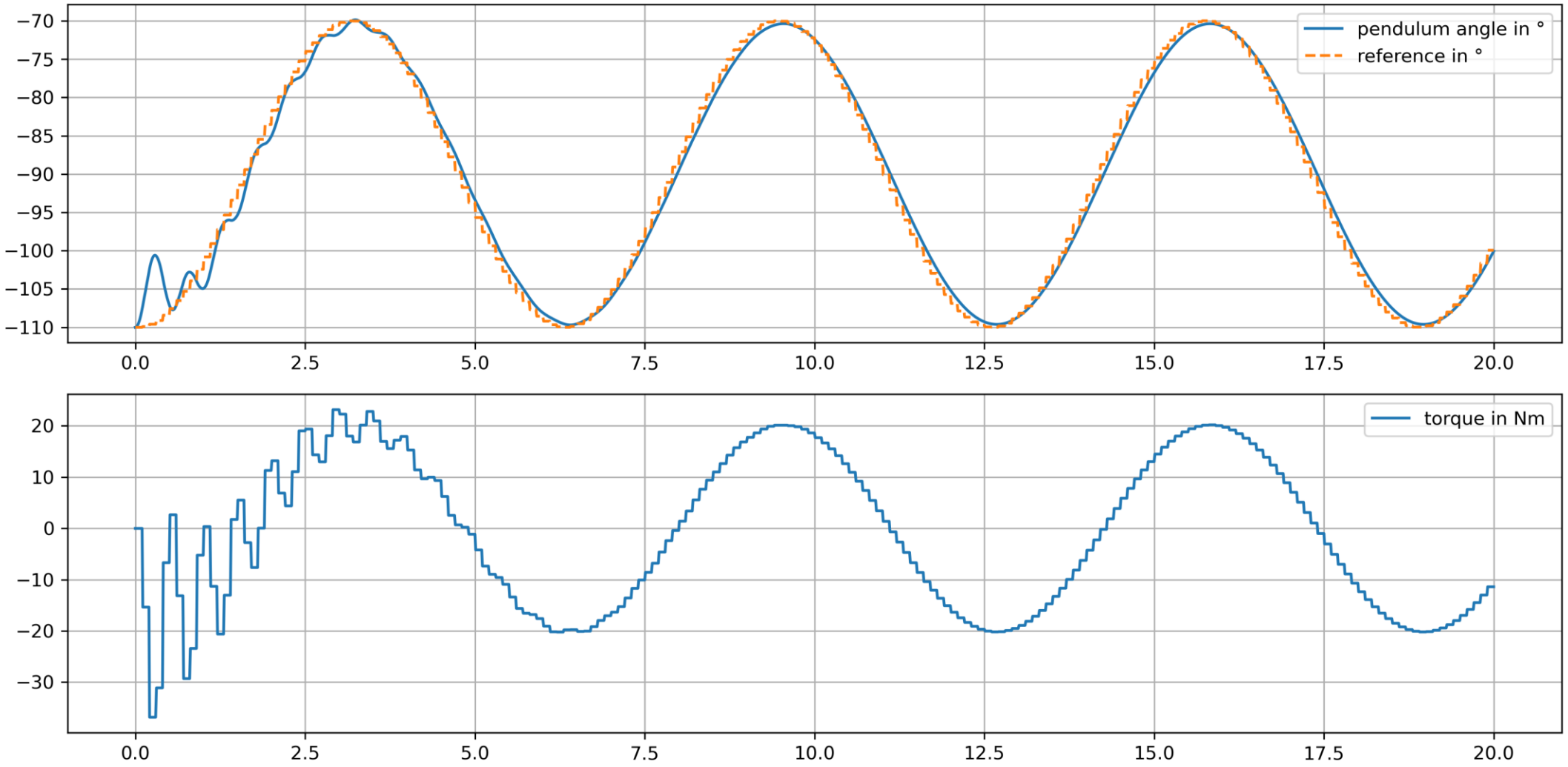
Controller Example

- We use a stranded built-in minimizer.
- Crucial for a god opisthion result is a good starting point.
- Choose appropriate bounds for your parameter to get stability.

```
# Initial values for the PID parameters (Kp, Ki, Kd)
initial_params = [0, 500, 29]

# Bounds for the PID parameters
bounds = [(0, 500), (0, 500), (0, 500)]

total_error = 0
total_torque = 0

# Optimization of the PID parameters
result = minimize(cost_function, initial_params, bounds=bounds, method='L-BFGS-B')

# Optimized PID parameters
Kp_opt, Ki_opt, Kd_opt = result.x
print(f"Optimized parameters: Kp={Kp_opt}, Ki={Ki_opt}, Kd={Kd_opt}")

t_sol, y_sol, ref_sol, torque_sol = simulation([Kp_opt, Ki_opt, Kd_opt], True)
```

Controller Example

```
Optimized parameters: Kp=99.40409874943714, Ki=500.0, Kd=29.052406226242955
```



Controller Example - Sensor

- We extend our system by a potentiometer to measure the angel
- The function takes the ideal angle computes the voltage of the measuring bridge like we du in the previse chapter
- it adds Gaussian distributed noise to the signal which we can control via the standard deviation parameter
- Finally, it quantized the signal according the ADC resolution bits
- The function works for scalars an arrays

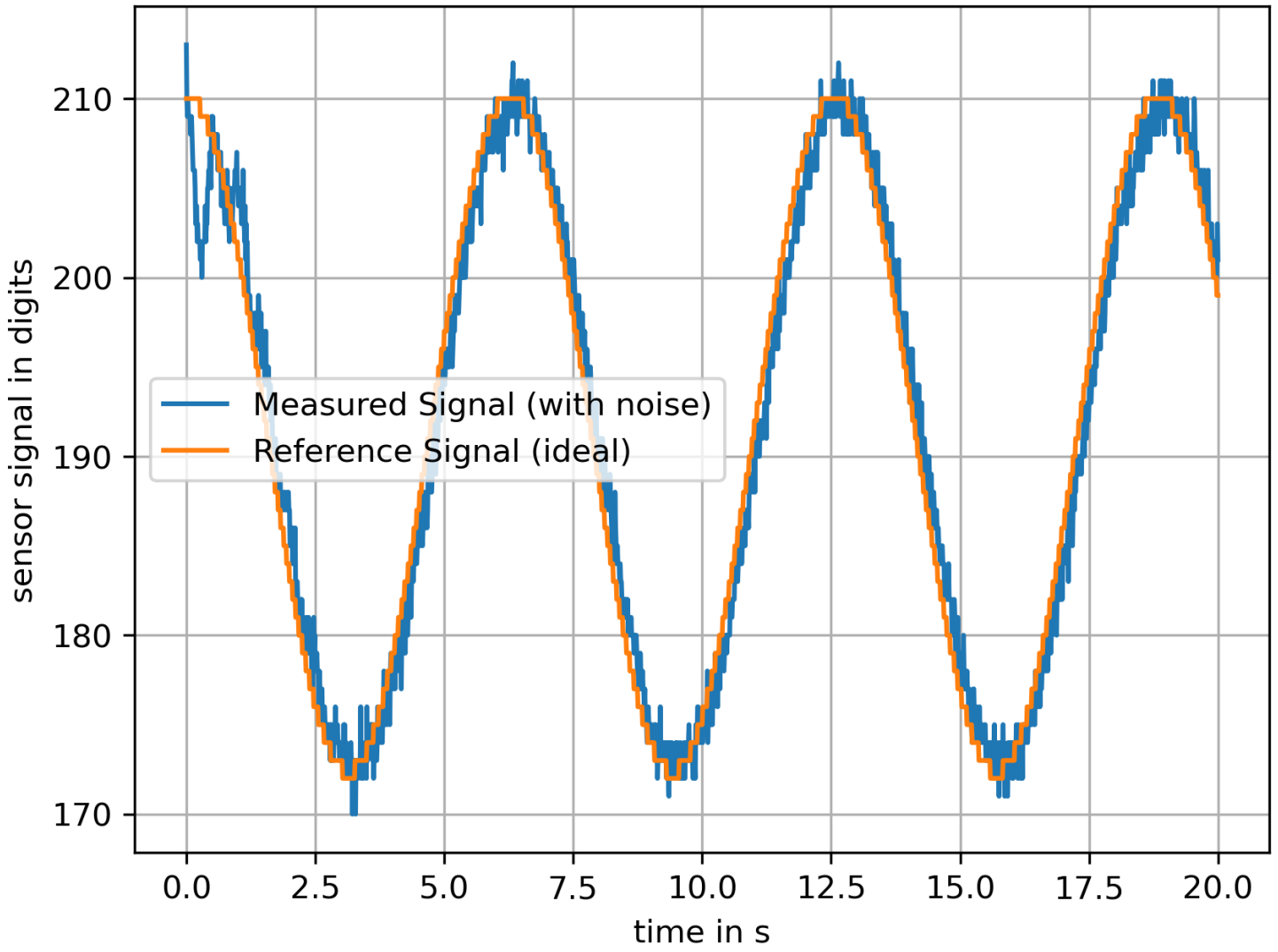
```python
U_0 = 3 # V
R_pot = 60000 # Ohm
R3 = 30000 # Ohm
R4 = 10000  # Ohm
alpha_max = 270 * np.pi/180 # °
U_ADC = 3  # V
n_bits_ADC = 8
```

```python
def sensor(q, noise_std):

    # Check if q is a scalar
    is_scalar = np.isscalar(q)
    q = np.atleast_1d(q)  # Converts q into an array if it's a scalar

    alpha = q + np.pi/2
    U_a = U_0 * (3/4 - alpha/alpha_max)

    # Add Gaussian noise
    mean = 0  # Mean of the noise
    noise = np.random.normal(mean, noise_std, size=U_a.shape)
    # Overlay signal with noise
    U_a_noisy = np.array(U_a + noise)

    U_a_noisy[U_a_noisy < 0] = 0
    U_a_noisy[U_a_noisy > U_ADC] = U_ADC

    # Number of quantization levels
    Q = 2**n_bits_ADC
    # Quantization: scale values, round them, bring back to original range
    delta = U_ADC / (Q - 1)
    U_a_quantized = delta * np.round((U_a_noisy) / delta)
    U_a_quantized = np.round((U_a_noisy) / delta)

    # If the input was a scalar, return a scalar
    if is_scalar:
        return U_a_quantized[0]
    else:
        return U_a_quantized
```

Controller Example - Sensor

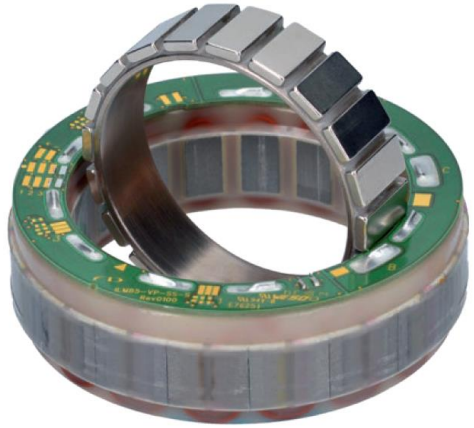
- We test our function with the computed angle and the reference angel form the last solution.

```
U_alpha = sensor(np.array(y_sol).T[1], 0.01)
U_ref = sensor(reference_trajectory(t_sol), 0)
```



Controller Example - Drive

- We further extend our system by a model of the drive.
- First, we use a quasi-static model
- I chose the ILM 50x08 motor from TQ systems
- We assume this brushless drive operates in star parallel configuration and is controlled like a brushed DC motor
- We can approximate the speed constant by the no load speed over the rated voltage.



BASIC DATA

	ILM-E50×08
Power [W]	203
Rated voltage U_r* [V]	48
Rated torque T_r* [Nm]	0.3
Peak torque T_{max} at 20% deviation from linearity [Nm]	0.98
Max rotation speed n_{max}** at U_r [rpm]	12,916***
Diameter D [mm]	50
Length L [mm]	17.25
Weight m [g]	76
Number of pole pairs	10
Rotor inertia J [kgcm²]	0.056

STAR PARALLEL

	ILM-E50×08
Rated current I_r* [A]	10
Copper losses P_{Lr} at T_r and 20°C [W]	11.3
Torque constant k_T* at 20°C [mNm/A]	30
Motor constant kM at 20°C [Nm/√W]	0.089
Terminal resistance R_{TT}* at 20°C [mΩ]	151
Terminal inductance L_{TT}* [µH]	121
No load speed [rpm]	12,916***

```
k_M = 29e-3  # Nm/A (Torque constant)
k_n = 12000/48  # RPM/V (Speed constant)
U_M = 16  # V (Motor voltage)
L_M = 123e-6  # H (Inductance)
R_M = 135e-3  # Ohms (Resistance)
i_M = 60  #  (gear ratio)
eta_M = 0.85  # Efficiency

def drive_qstat(U, omega):
    n = i_M * omega * 30 / np.pi  # Rotational speed in RPM
    M = eta_M * i_M * k_M / R_M * (U - n / k_n)  # Torque
    return M
```

Controller Example +qstat drive +sensor

- The function system_equations assembles now the drive and pendulum equations.
- The function drive_qstat ads an algebraic aquation to compute the torque of the drive.
- The torque is not state of an ODE system.
- The control variable u_control is a value between -1 and 1 and represents the duty cycle and control direction of a pulse wide modulation.
- u_control varies the motor current from -16 to 16 V

```python
def pendulum(t, y, M):
    omega, q = y
    domega = - (g / L) * np.cos(q) + M / (m * L**2)
    dq = omega
    return [domega, dq]


def drive_qstat(u_control,omega):
    U = U_M * u_control
    n = i_M * omega * 30 / np.pi  # rotational speed 1/min
    M = eta_M * i_M * k_M / R_M * (U - n / k_n)
    return M


# Function to create the right-hand side of the equation
def system_equations(t, y, args):
    M = drive_qstat(args, y[0])
    dydt_pendulum = pendulum(t, y, M)
    return dydt_pendulum
```

Controller Example +qstat drive +sensor

- To integrate our potentiometer, we modifies the controller call in the for loop.
- With the senor function we map the angle of the pendulum and the reference angle to a digital voltage value between 0 and 256 according to our 8-bit ADC.
- For the sensor signal we add noise
- The control output is bounded to its max and min value.

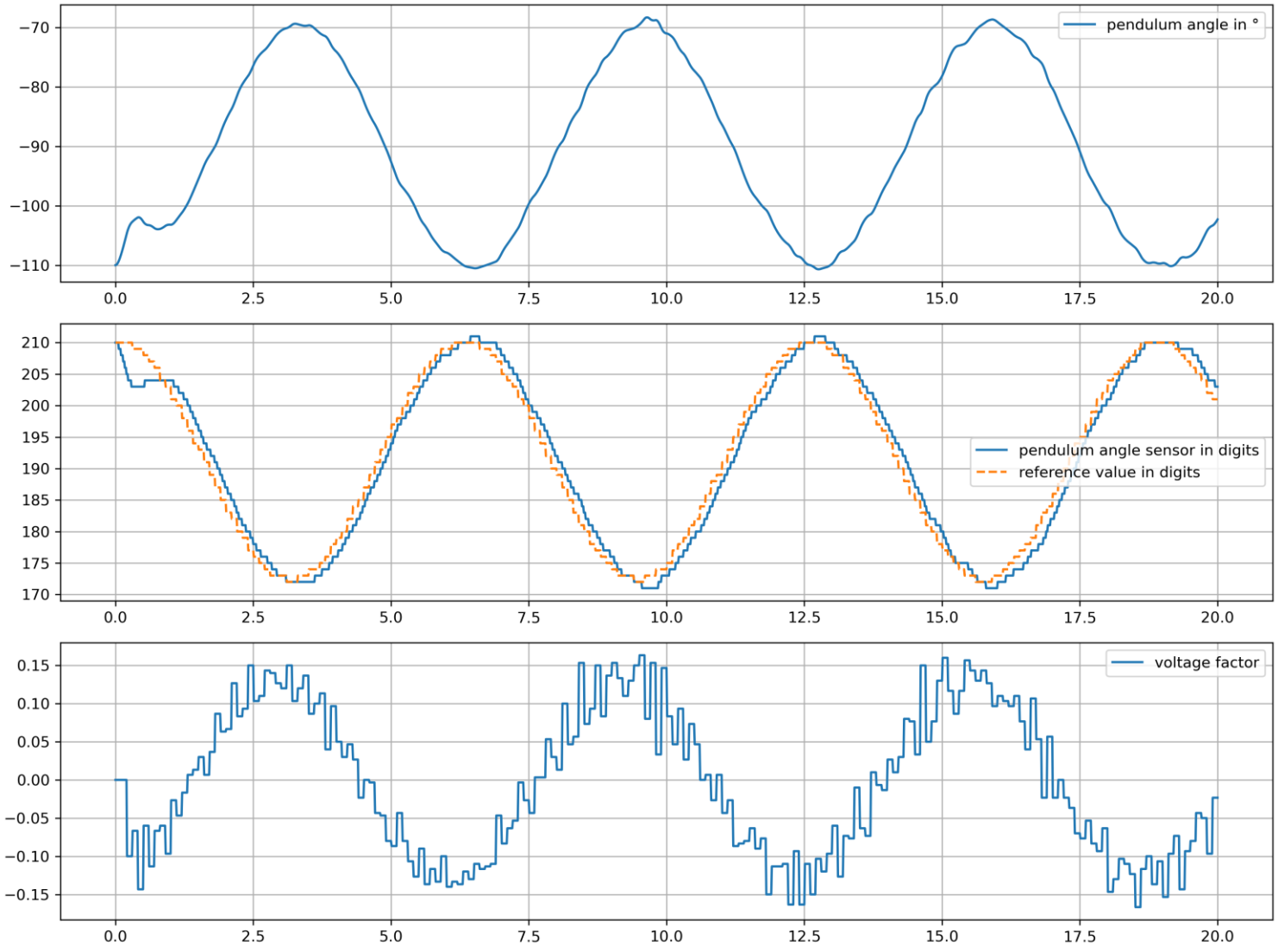
```
for i in range(num_steps):
    current_time = t_control[i]

    # Reference trajectory and measurement
    reference = sensor(reference_trajectory(current_time), 0)  # generate digital reference signal
    measured = sensor(current_state[1], 0.01)

    # Calculate control signal
    u_control = controller.compute_control(-reference, -measured)
    if u_control > 1: u_control = 1
    if u_control < -1: u_control = -1
```

Controller Example +qstat drive +sensor

- To solve our system, we call the solver like bevor with u_control as hand over argument.
- We see that now the RK4 solver leads to extreme high computational times.
- By adding the drive model we get a stiff system. That means the difference between the eigenvalues of the Jacobian matrix differs strongly.
- To solve stiff systems, we use am implicit solver, like det BDF-Solver (Backward Differentiation Formula)

```
# Solve the ODE for the current interval, passing the torque
sol = solve_ivp(system_equations, t_span, current_state, args=(u_control,), method='BDF', t_eval=t_eval, dense_output=True)
```

Controller Example +qstat drive +sensor

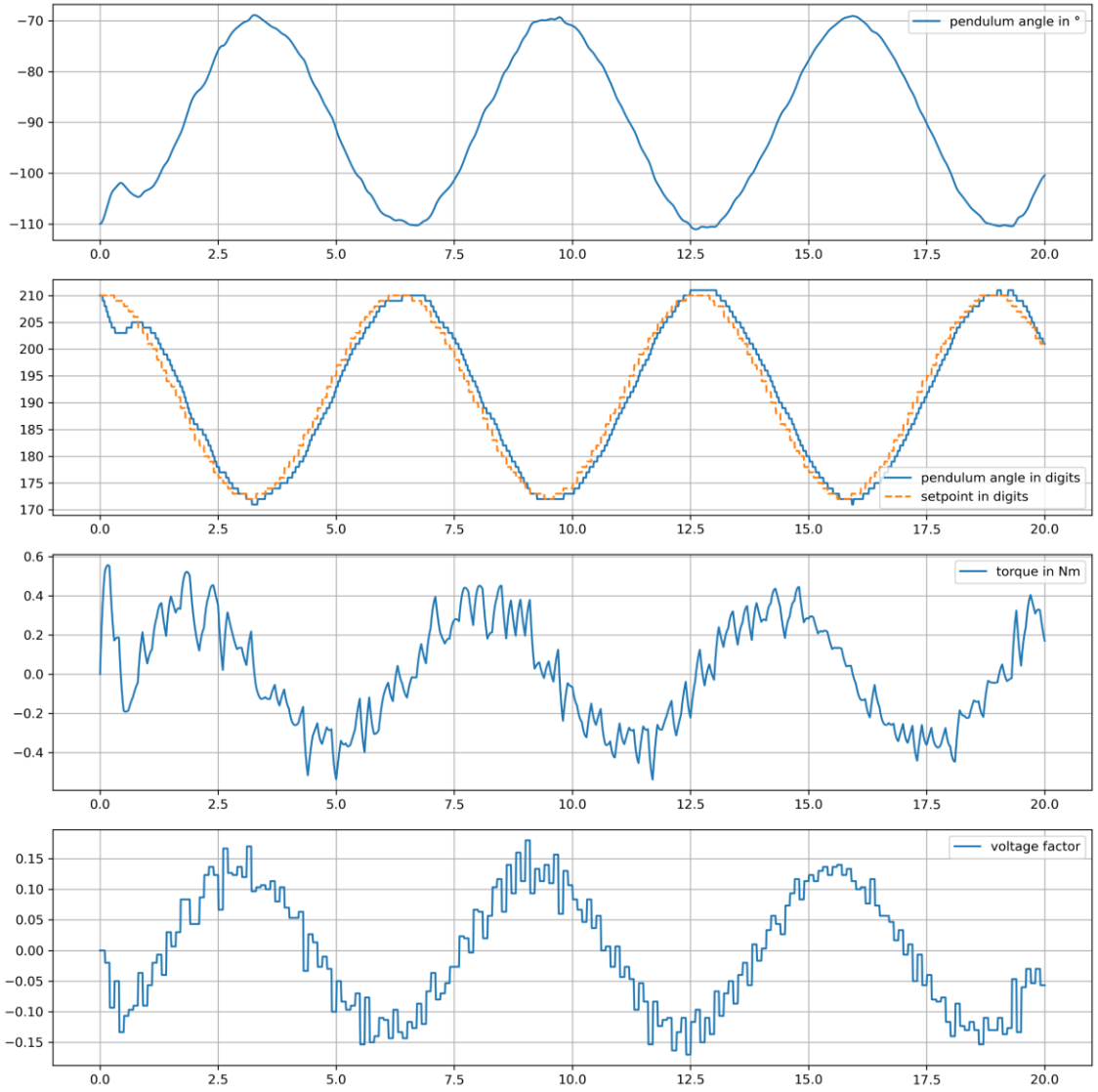


Controller Example +drive +sensor

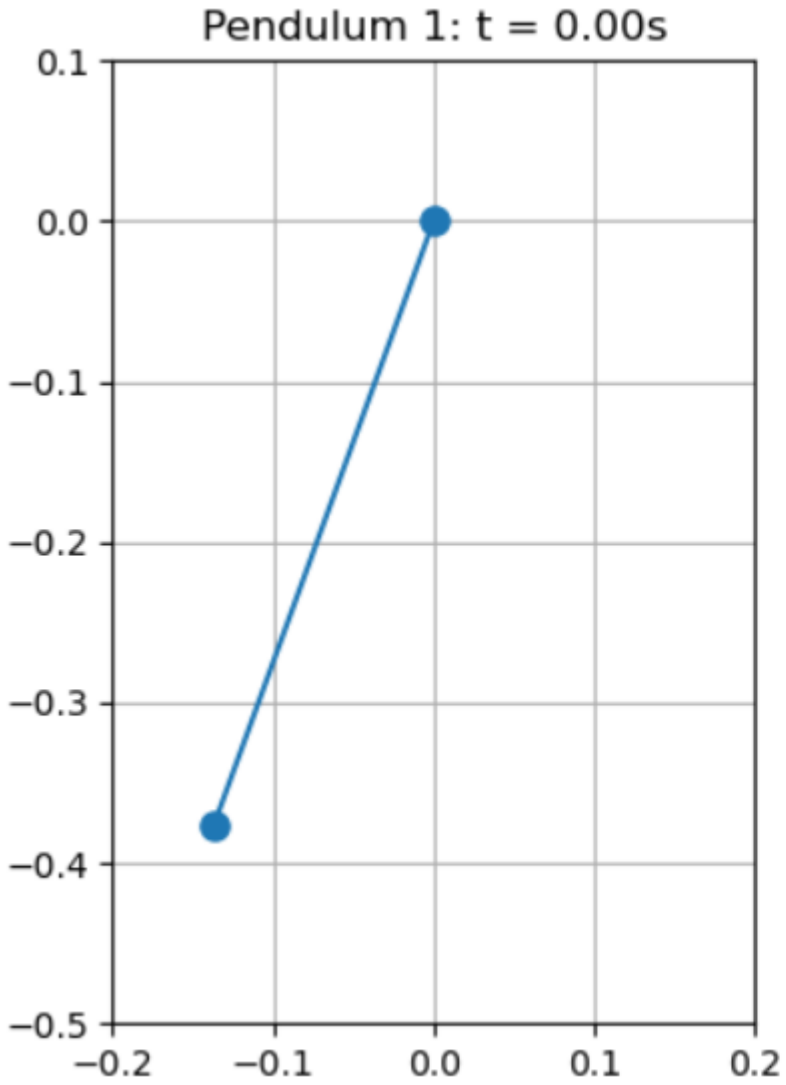
- Finaly, we look a full dynamic model for the drive.
- We solve the torque M as a state of our ODE system.
- The function system equations again assembles the sub models and returns the derivative of all states for the solver.
- The for loop and the solver doesn't change.
- We just had to consider that the solver returns the solution of all states omega, q and M.

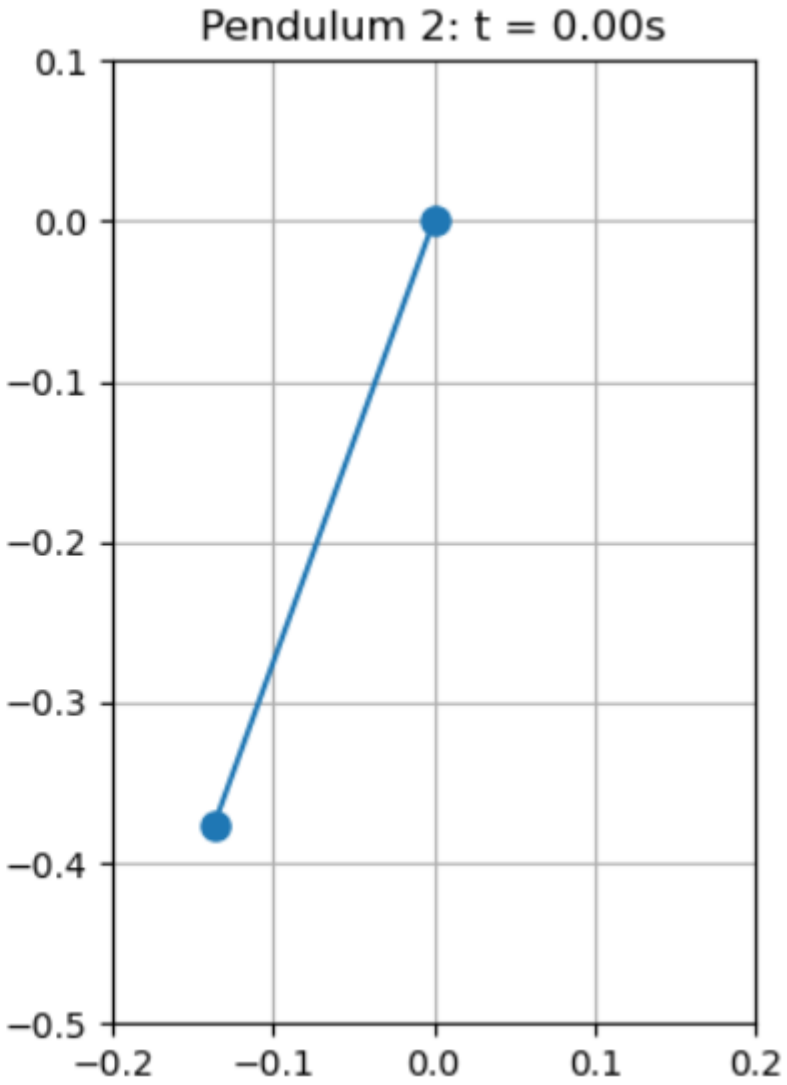
```
def pendulum(t, y, M):
    omega, q = y
    domega = - (g / L) * np.cos(q) + M / (m * L**2)
    dq = omega
    return [domega, dq]


def drive(t, y, u_control, omega):
    M = y
    U = U_M * u_control
    n = i_M * omega*30/np.pi # rotational speed (1/min)
    I = M / (eta_M * i_M * k_M)
    dI = 1/L_M * ( U - I*R_M - n/k_n )
    dM = eta_M * i_M * k_M * dI
    return [dM]


# Function to create the right-hand side of the equation
def system_equations(t, y, args):
    dydt_pendulum = pendulum(t, y[0:2], y[2])
    dydt_drive = drive(t, y[2], args, y[0])
    return dydt_pendulum + dydt_drive
```

Controller Example +drive +sensor



Controller Example animation





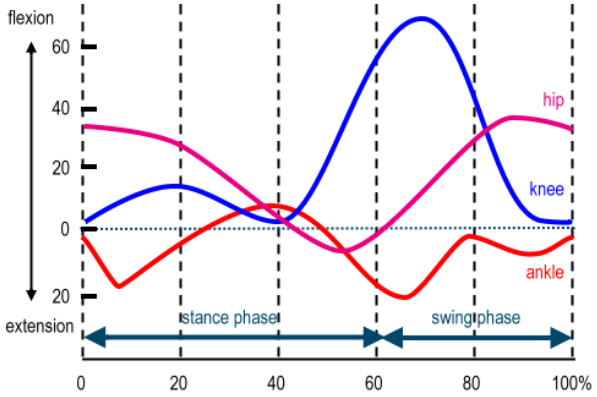
High level control

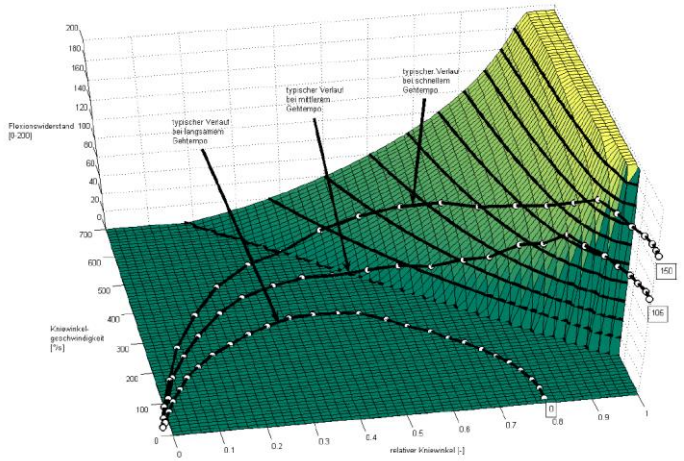
High Level Control

- The high level control acts on way more abstract lever.
- It determine the user intension out of sensor data and translate it to set points for the low level control.

evaluating signal data → Identifying the situation → Adjusting nominal values

Adjusting set points





Nervous system/control
High Level Control

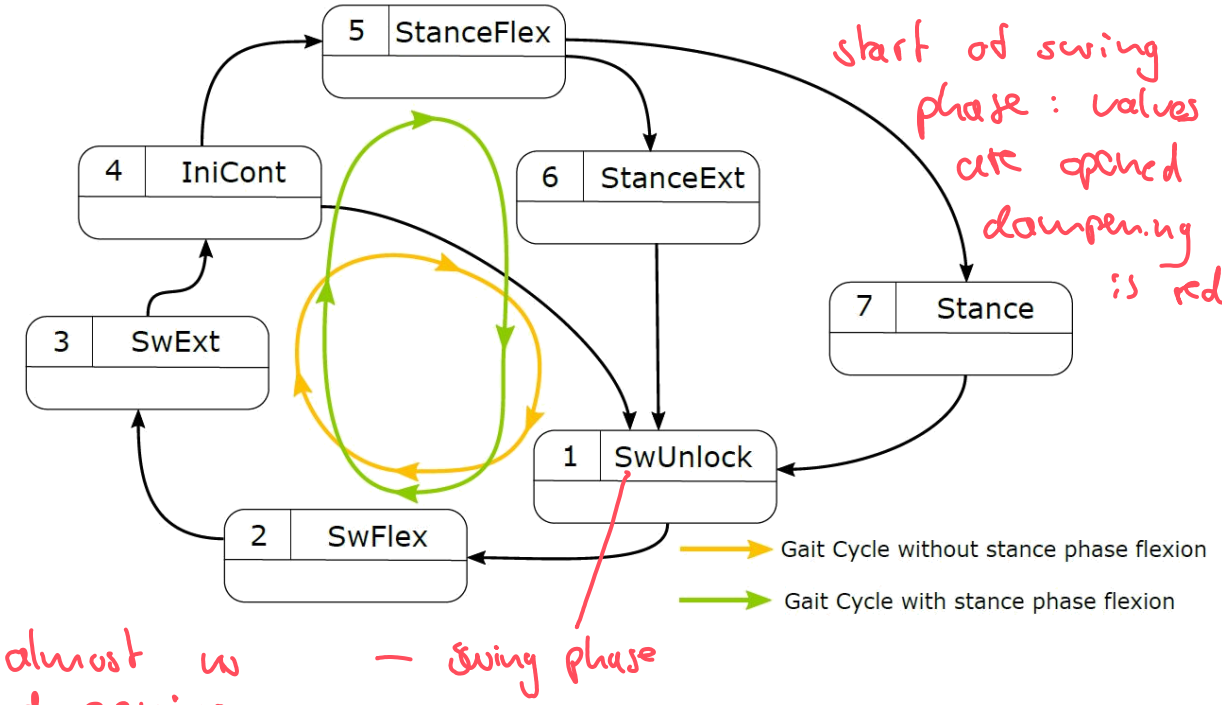
State machine.
Basic control of a microprocessor knee joint

- Discrete states
- Transition between states Depending on conditions
- Execution of program code within a state depending on history
  - upon admission
  - during disposition
  - on withdrawal
  - time-controlled

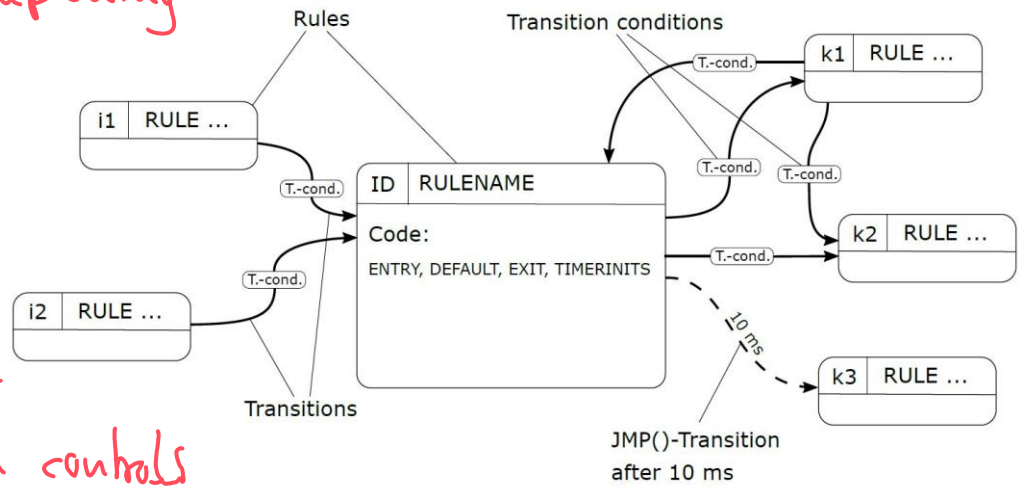
  } rules
- Execution of the state machine in the system clock (synchronous, standard = 10 ms)
- sequential execution of up to 4 rulesets per cycle (MemSets)

slower than controls

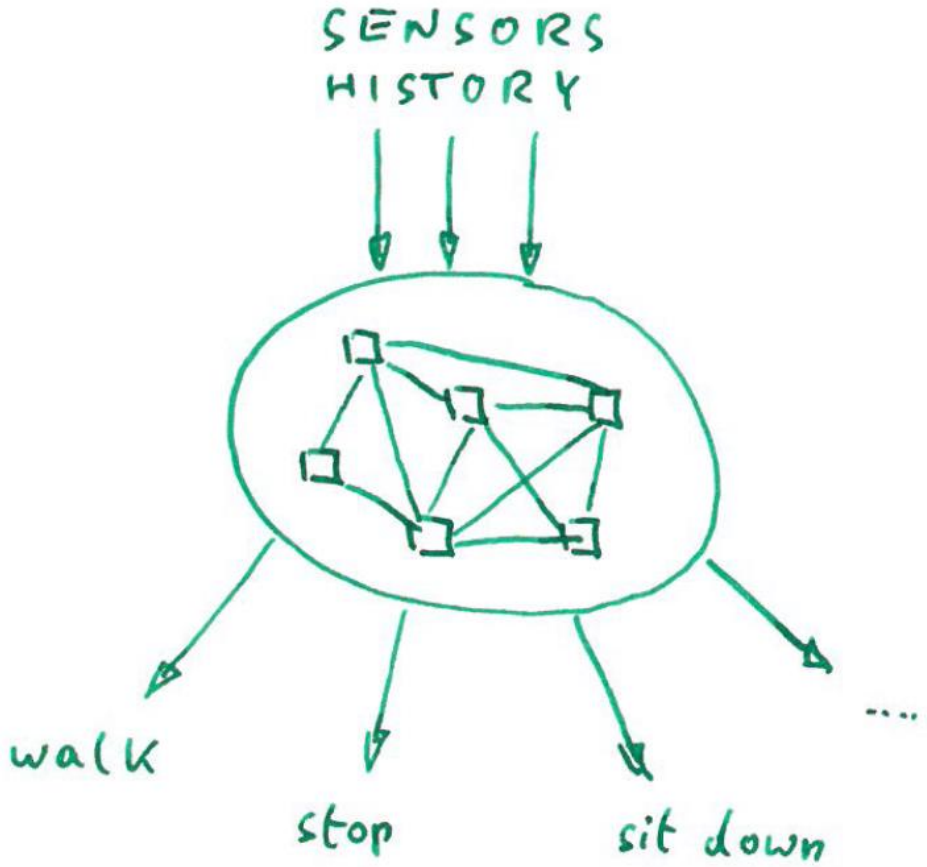
critical moment: start of swing phase: valves are opened dampening is reduced



almost no dapening — swing phase



200.000.000.000 decisions to take



1 out of a million decisions is wrong

200.000 missed steps

20.000 falls

2.000 injuries

Nervous system/control Fields of research

→ predict. St for product admission
↳ safety critical → classical predictable modelling
↓
testing & validation

Currently, due to reproducibility and stability, focus on conventional program structures such as state machine.
Machine learning is becoming increasingly important, especially for non-safety-critical functions (e.g. comfort topics). A large number of studies have been conducted in this area in the field of research.

Myo Plus is an Ottobock product that already uses this form of algorithm to control hand prostheses.

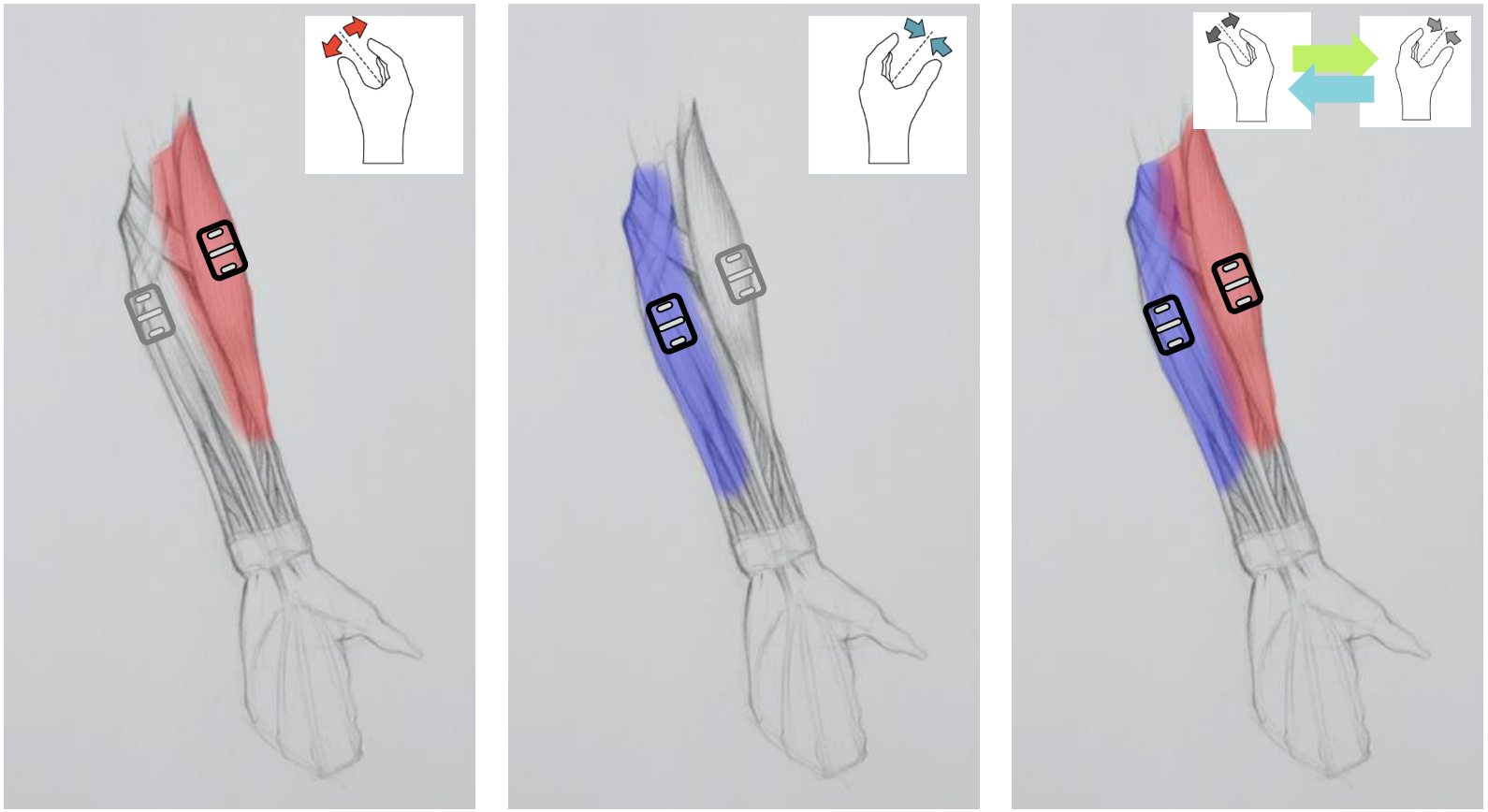
Nervous system/control
Myo control

upper extremities : no cyclic movement
↳ completley arbitrary

This conventional way of controlling prosthetic hands hasn't changed in decades (1940).

We have one electrode on the inside and one on the outside of the arm. So with these two muscles, I can open and close the hand, and switch between different modes, joints and grips with a single contraction.

What we actually want to do is control 14 different grips intuitively.

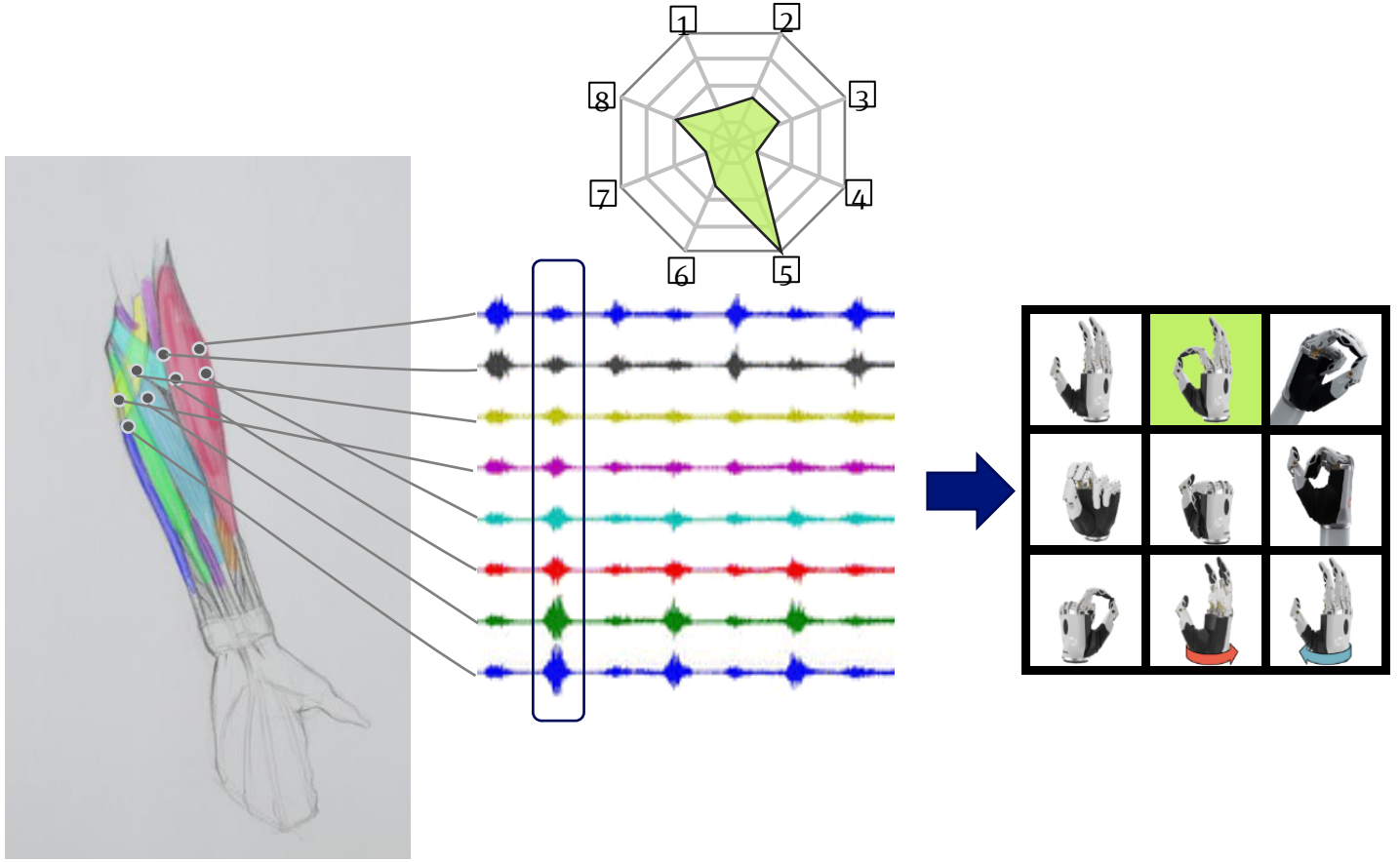


Nervous system/control
Myo Plus pattern recognition

Using machine learning/pattern recognition opens up a whole range of possibilities for the user. With conventional control devices, the user has to learn to control the prosthesis. With machine learning, the prosthesis adapts to the user, resulting in intuitive operation.

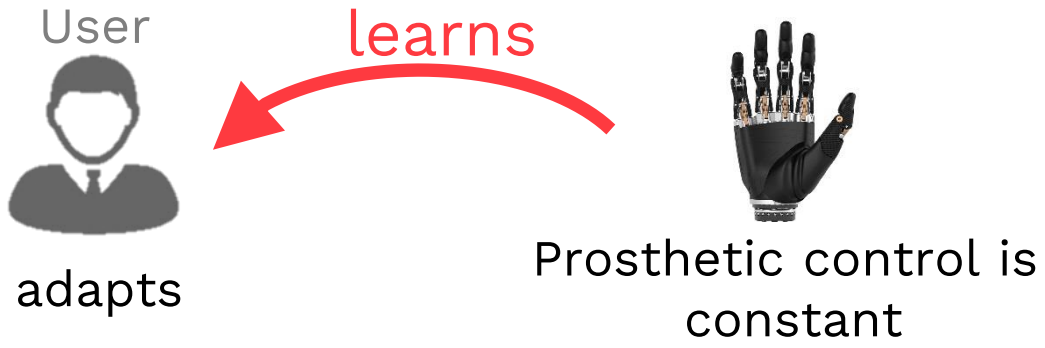
Due to its robustness and 100 % predictable behavior, a linear discriminant analysis (LDA) is used as an algorithm.

Eight electrodes are used to obtain a holistic picture of the muscle signals in the arm. In order to collect the necessary data for training, we practise supervised learning using a tablet app.

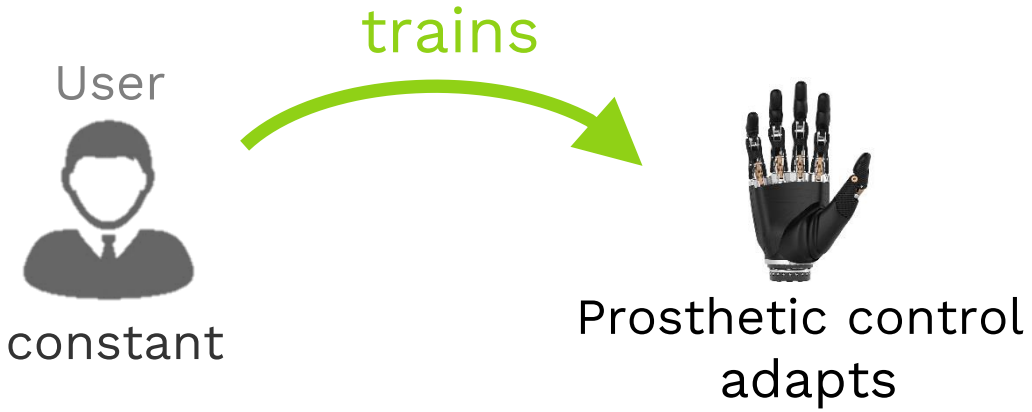


Control of hand prosthesis

Conventional control



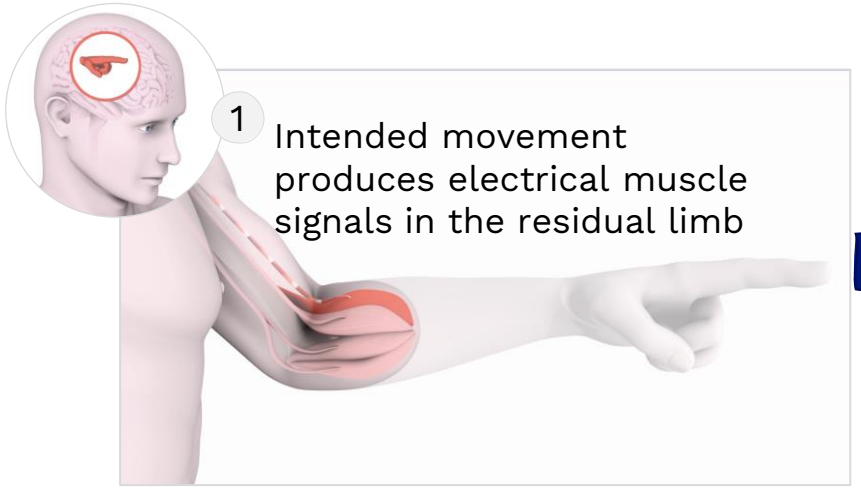
Myo Plus Pattern recognition



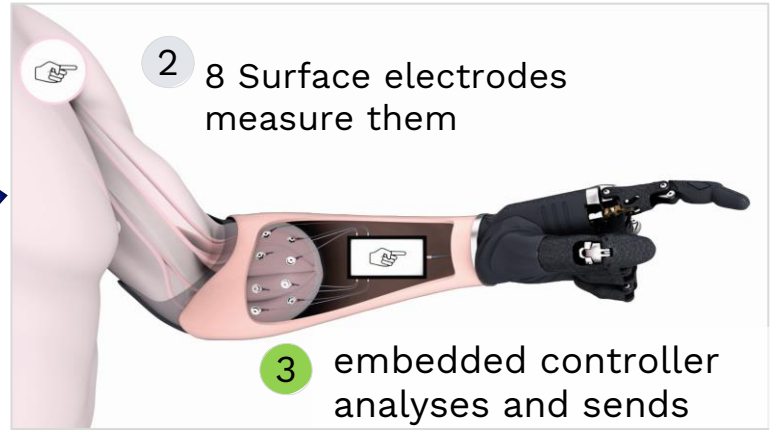
Myo Plus – man-machine interface

- Myo Plus is paradigm shift in prosthesis control
- User trains his personal muscle movements patterns to the system
- he can on his own enhance and train new grips at anytime
- APP is used as interface and “window” into the prosthesis
- Embedded controller is the “brain” of the system
- Cloud connection enables remote services

Machine learning to train and determine the intended user movements



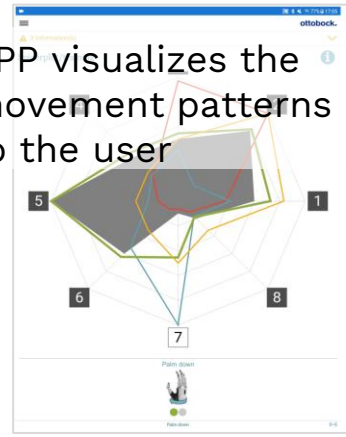
1 Intended movement produces electrical muscle signals in the residual limb

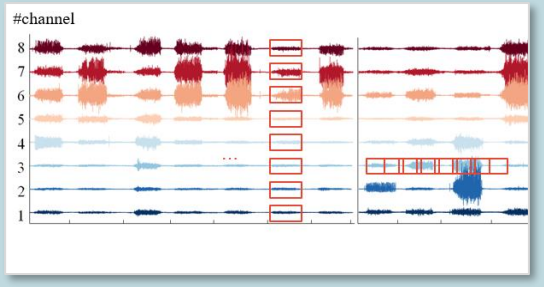


2 8 Surface electrodes measure them

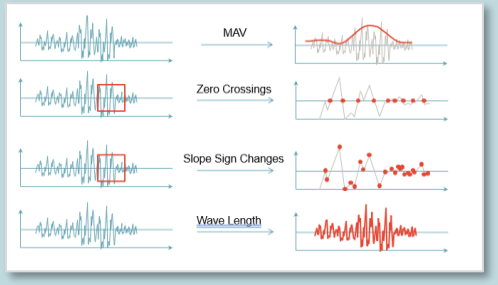
3 embedded controller analyses and sends command to hand

4 APP visualizes the movement patterns to the user

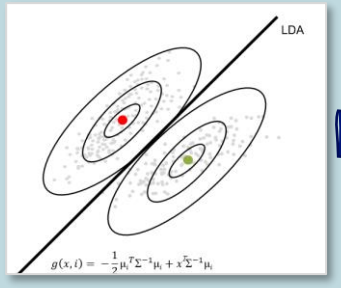




2 muscle signals are segmented



3 signal processing's methods prepare data



3 modelling with machine learning algorithms

4 from User trained classifier decides current movement

Nervensystem/Steuerung
Control obere Extremität

Myo Plus - The next generation of prosthetic control – YouTube

Ottobock Myo Plus pattern recognition: The AI control for myoelectric arm prostheses | Ottobock – YouTube